

Functional API: Trích xuất Tính năng (Feature Extraction)

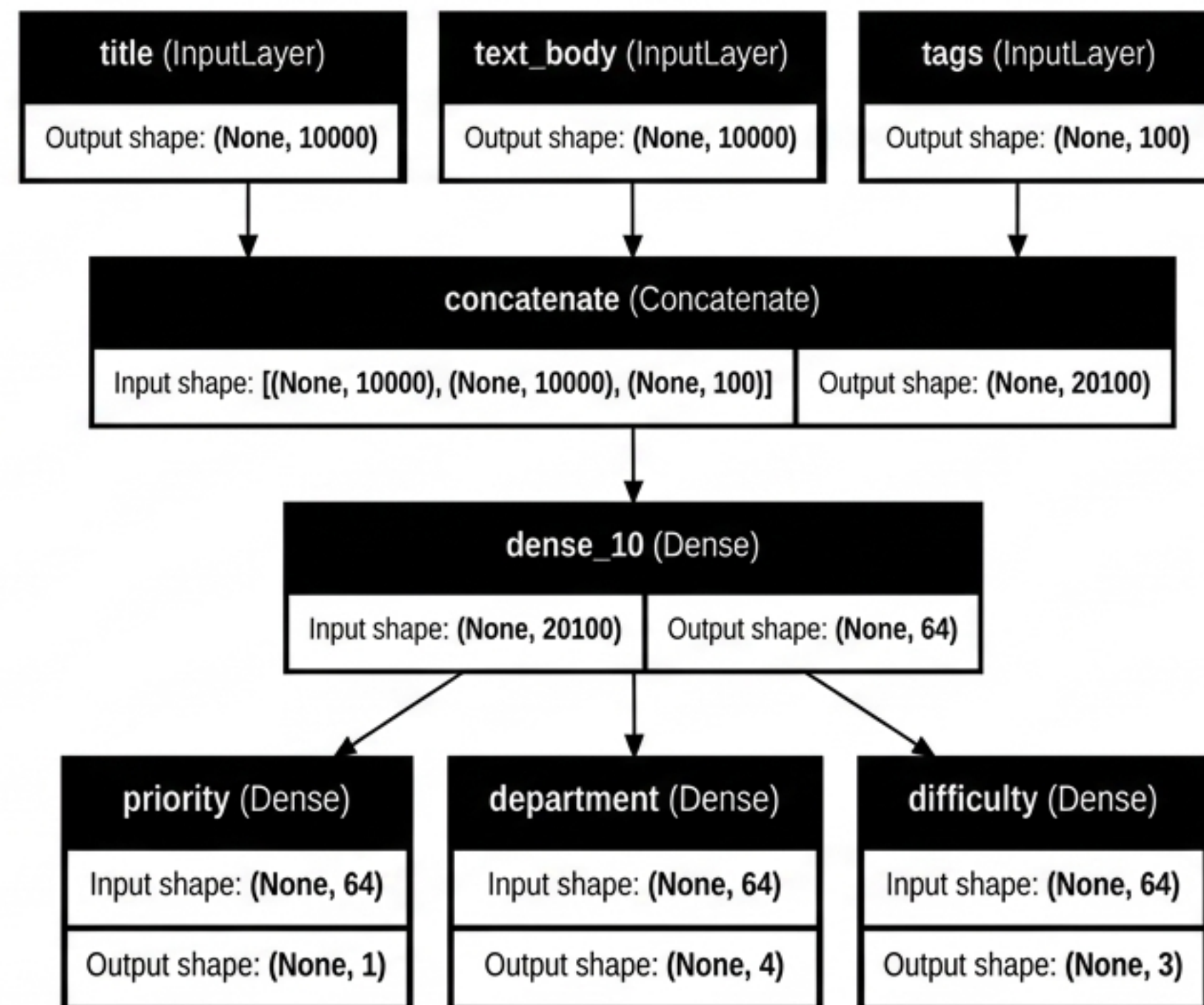


Mô hình chứa các nút trung gian (Intermediate Nodes). Ta có thể tái sử dụng chúng.

Trích xuất layer tại mốc số 4 (**Dense layer**) và bổ sung 1 nhánh difficulty (độ khó).

```
features = model.layers[4].output
difficulty = layers.Dense(
    3, activation="softmax", name="difficulty"
)(features)

new_model = keras.Model(
    inputs=[title, text_body, tags],
    outputs=[priority, department, difficulty]
)
```



Hình 7.5: Mở rộng đồ thị

Phương pháp 3: Model Subclas-sing



Đặc điểm

Phương pháp cấp thấp, linh hoạt nhất.



Bản chất

Kế thừa trực tiếp Lập trình Hướng đối tượng (OOP) từ lớp keras.Model.



Khởi tạo (__init__)

Khai báo các Layer (chứa trọng số) tại hàm tạo.



Tính toán (call)

Định nghĩa dòng chảy Toán học ngầm (Forward pass) tại hàm này.

Ưu điểm: Cung cấp khả năng kiểm soát tuyệt đối, ví dụ sử dụng vòng lặp for, đệ quy bên trong mạng Nơ-ron.

Model Subclassing: Mã nguồn

Viết lại Ticket Classifier dưới dạng Class:



```
class CustomerTicketModel(keras.Model):
    def __init__(self, num_departments):
        super().__init__() # Always call parent constructor
        self.concat_layer = layers.Concatenate()
        self.mixing_layer = layers.Dense(64, activation="relu")
        self.priority_scorer = layers.Dense(1, activation="sigmoid")
        self.department_classifier = layers.Dense(num_departments, activation="softmax")

    def call(self, inputs): # Forward pass logic
        title = inputs["title"]
        text_body = inputs["text_body"]
        tags = inputs["tags"]

        features = self.concat_layer([title, text_body, tags])
        features = self.mixing_layer(features)
        priority = self.priority_scorer(features)
        department = self.department_classifier(features)
        return priority, department
```

Model Subclassing: Cảnh báo



Nhược điểm: Bạn đang viết "**mã byte**" dạng **hộp đen** (Black-box), không còn là đồ thị (Graph) rõ ràng nữa.

- ✗ Không thể gọi hàm `summary()` để in ra cấu trúc các nút kết nối.
- ✗ Không thể sử dụng `plot_model()` để vẽ ảnh cấu trúc mạng.
- ✗ Không thể dùng kĩ thuật Feature Extraction do không thể truy xuất `model.layers[i].output`.

Người lập trình hoàn toàn chịu trách nhiệm cho các lỗi logic bên trong `call()`.

Tùy biến Keras: Khi các hàm có sẵn là không đủ

API `model.fit()` rất tốt cho hầu hết bài toán thực tế. Nhưng với các nhà nghiên cứu, họ thường phát minh ra các hàm Mất mát (Losses) mới, thước đo (Metrics) mới, hoặc những thuật toán Huấn luyện phi tiêu chuẩn (GANs).



Tự định nghĩa Hàm Mất mát (Custom Losses)



Đôi khi hàm Mean Squared Error (MSE) mặc định là không đủ.

Viết một hàm (function) đơn giản:

```
import tensorflow as tf

def custom_mean_squared_error(y_true, y_pred):
    return tf.reduce_mean(tf.square(y_true - y_pred))

model.compile(optimizer="adam", loss=custom_mean_squared_error)
```

Hoặc kế thừa lớp keras.losses.Loss:

```
class CustomMSE(keras.losses.Loss):
    def __init__(self, name="custom_mse", **kwargs):
        super().__init__(name=name, **kwargs)

    def call(self, y_true, y_pred):
        return tf.reduce_mean(tf.square(y_true - y_pred))
```

Tự định nghĩa Thước đo (Custom Metrics)



Sự mang Trạng thái - Stateful

Loss	Metric
Tính cục bộ trên mỗi Batch (Lô).	Phải cộng dồn trên nhiều Batch và tính trung bình trong suốt 1 Epoch.

Kế thừa lớp `keras.metrics.Metric` yêu cầu 3 hàm:

1. `update_state()`

Cộng dồn kết quả đếm cho mỗi Batch.



2. `result()`

Tính toán và trả về về kết quả hiển thị cho người dùng.



3. `reset_state()`

Xóa sạch bộ đếm về 0 khi bắt đầu một Epoch mới.

Custom Metrics: Cài đặt thực tế (Root Mean Squared Error)



```
class RootMeanSquaredError(keras.metrics.Metric):  
    def __init__(self, name="rmse", **kwargs):  
        super().__init__(name=name, **kwargs)  
        # Declare state variables  
        self.mse_sum = self.add_weight(name="mse_sum", initializer="zeros")  
        self.total_samples = self.add_weight(name="total_samples", initializer="zeros")  
  
    def update_state(self, y_true, y_pred, sample_weight=None):  
        mse = tf.reduce_mean(tf.square(y_true - y_pred))  
        self.mse_sum.assign_add(mse)  
        self.total_samples.assign_add(1.0)  
  
    def result(self):  
        # Return sqrt of accumulated mean MSE  
        return tf.sqrt(self.mse_sum / self.total_samples)  
  
    def reset_state(self):  
        self.mse_sum.assign(0.)  
        self.total_samples.assign(0.)
```

Cập nhật trạng thái: Cộng dồn MSE và số lượng mẫu sau mỗi Batch.

Tính toán kết quả: Trả về căn bậc hai của MSE trung bình tích lũy.

Đặt lại trạng thái: Xóa sạch các biến trạng thái về 0 khi bắt đầu một Epoch mới.

Viết lại Vòng lặp Huấn luyện với GradientTape

Thay vì gọi **model.fit()**, ta có thể sử dụng **tf.GradientTape** để tự điều khiển phép tính Đạo hàm riêng (Backpropagation):

$$\Delta W = -\eta \cdot \frac{\partial L}{\partial W} \quad (1)$$

Quy trình 4 bước của 1 Batch (Lô dữ liệu):

4. Gửi Gradient cho Trình tối ưu hóa (Optimizer) để cập nhật Trọng số W .



3. Dùng Tape truy xuất Gradient $\partial \mathcal{L} / \partial W$ của các Trọng số.



1. Mở không gian bộ nhớ (Tape), truyền dữ liệu qua Mạng.



2. Tính toán Loss $\mathcal{L}$.





Custom Training Loop: Mã nguồn GradientTape

Đây là kiến thức quan trọng nhất dành cho một Chuyên gia Học sâu.

```
optimizer = keras.optimizers.Adam(learning_rate=1e-3)
loss_fn = keras.losses.SparseCategoricalCrossentropy()

for epoch in range(3): # Epoch loop
    for step, (x_batch, y_batch) in enumerate(train_dataset): # Batch loop
        # 1. Open Tape
        with tf.GradientTape() as tape:
            # 2. Feed-forward and calculate loss
            logits = model(x_batch, training=True)
            loss_value = loss_fn(y_batch, logits)

            # 3. Compute Gradients
            gradients = tape.gradient(loss_value, model.trainable_weights)

            # 4. Apply Gradients (Update weights)
            optimizer.apply(gradients, model.trainable_weights)
```

Ghi đè hàm `train_step()`



Để kết hợp vòng lặp thủ công với tính năng của **`fit()`** (hiển thị thanh tiến trình), bạn có thể ghi đè **`train_step()`**:

```
class CustomModel(keras.Model):
    def train_step(self, data):
        inputs, targets = data
        with tf.GradientTape() as tape:
            predictions = self(inputs, training=True)
            loss = self.compute_loss(y=targets, y_pred=predictions)

        gradients = tape.gradient(loss, self.trainable_weights)
        self.optimizer.apply(gradients, self.trainable_weights)

        # Update Metrics
        for metric in self.metrics:
            if metric.name == "loss":
                metric.update_state(loss)
            else:
                metric.update_state(targets, predictions)
        return {m.name: m.result() for m in self.metrics}
```

Bạn vẫn có thể dùng `model.fit()` và tận hưởng mọi Callback.

Sức mạnh của Callbacks



Vấn đề

Huấn luyện mô hình Deep Learning có thể **tốn hàng giờ hoặc nhiều tuần**. Bạn không thể cứ ngồi nhìn màn hình suốt quá trình đó để can thiệp kịp thời.

Giải pháp

Callbacks là các "hook" (chốt chặn) được **kích hoạt tự động** vào các thời điểm cụ thể (Đầu/cuối Epoch, Đầu/cuối Batch).

Ứng dụng

Tự dừng sớm, Lưu trọng số, Thay đổi tốc độ học...

Các Callbacks tích hợp: EarlyStopping & ModelCheckpoint



EarlyStopping

Tự động dừng huấn luyện khi val_loss không còn giảm (Ngăn Overfitting).



ModelCheckpoint

Tự động sao lưu Trọng số Tốt nhất xuống ổ cứng.

```
callbacks_list = [  
    keras.callbacks.EarlyStopping(  
        monitor="val_accuracy",  
        patience=2, # Stop if no improvement after 2 epochs  
    ),  
    keras.callbacks.ModelCheckpoint(  
        filepath="my_checkpoint.keras",  
        monitor="val_loss",  
        save_best_only=True, # Only save model with lowest val_loss  
    )  
]  
model.fit(....., callbacks=callbacks_list, ...)
```

Tùy biến Callback (Custom Callback)



Bạn có thể tự tạo Callback bằng cách kế thừa `keras.callbacks.Callback`. Các hàm sự kiện bạn có thể ghi đè:



```
class MyCustomCallback(keras.callbacks.Callback):
    def on_epoch_begin(self, epoch, logs=None):
        print(f"Start epoch {epoch}")

    def on_epoch_end(self, epoch, logs=None):
        print(f"End epoch {epoch}")

    def on_batch_begin(self, batch, logs=None): ...
    def on_batch_end(self, batch, logs=None): ...
    def on_train_begin(self, logs=None): ...
    def on_train_end(self, logs=None): ...
```



Custom Callback: Ví dụ ghi lại Loss theo Batch

```
import matplotlib.pyplot as plt

class LossHistory(keras.callbacks.Callback):
    def on_train_begin(self, logs=None):
        self.per_batch_losses = [] # Store loss for every batch

    def on_batch_end(self, batch, logs=None):
        self.per_batch_losses.append(logs.get("loss"))

    def on_train_end(self, logs=None):
        plt.clf()
        plt.plot(range(len(self.per_batch_losses)), self.per_batch_losses,
                 label="Training loss")
        plt.xlabel(f"Batches (Total: {len(self.per_batch_losses)})")
        plt.ylabel("Loss")
        plt.legend()
        plt.savefig("loss_history_callback_example.png")
```